

Autonomous-QA Research: Containerized Emulators, Video Evidence, Vision Verification, Tracker Extraction

Generated: 2026-07-02 (research subagent)

Executive summary. For Lava's autonomous-QA loop on a Linux x86_64 host with /dev/kvm and rootless podman 5.7.1, the strongest KVM-accelerated container path is the Android SDK emulator packaged via Google's android-emulator-container-scripts (Apache-2.0, --device /dev/kvm, gRPC + adb-over-TCP), with budtmo/docker-android (MIT, noVNC + video) as a batteries-included alternative and redroid (Apache-2.0) as the no-KVM fallback for constrained runners; Google's official Cuttlefish (cvd) is the highest-fidelity self-hostable "device farm" primitive (KVM + vhost_vsock, on-prem Docker + Cloud Orchestrator). For evidence capture, scrcpy (Apache-2.0) beats adb screenrecord because it has no 3-minute cap (the cap is a hardcoded kMaxTimeLimitSec = 180 in the AOSP screenrecord binary; Maestro patches it via Radare2 binary rewriting), and ffmpeg segment muxer + the hash/framehash muxers give tamper-evident chunk+SHA-256 evidence. For AI verification, Midscene.js (MIT, drives Android over ADB, runs self-hostable VLMs UI-TARS/Qwen) and ByteDance UI-TARS (Apache-2.0, 2B/7B/72B) let a vision model assert user-visible outcomes from frames. For legal authorized-testing extraction of .torrent/magnet links from RuTracker/RuTor/NNM-Club/Kinozal behind Cloudflare + DDoS-Guard, TorAPI (MIT, Node.js, already targets exactly these four trackers), curl_cffi (MIT, JA3/JA4/HTTP2/HTTP3 impersonation), and challenge-solver proxies Byparr (Camoufox) / FlareSolverr (deprecated) are the concrete OSS building blocks.

1. Containerized Android emulators with KVM under rootless podman

Baseline facts about the host target: Cuttlefish and the SDK emulator both require the host to expose KVM; Cuttlefish additionally needs the `vhost_vsock` and `vhost_net` kernel modules for guest I/O and networking (source.android.com/docs/devices/cuttlefish/get-started). The standard container passthrough for the SDK emulator is `docker run --device /dev/kvm ...` (github.com/google/android-emulator-container-scripts).

Rootless podman /dev/kvm caveat (verified): rootless podman can silently fail to mount `/dev/kvm` when it is accessed through group membership; the documented workaround is to bind-mount the node with `-v /dev/kvm` instead of `--device /dev/kvm` (containers/podman#16701), corroborated by the Red Hat rootless-KVM/QEMU guidance (developers.redhat.com/articles/2024/12/18/rootless-virtual-machines-kvm-and-gemu). ACTION for Lava's Containers submodule: the `containerized.go` path that hard-requires `--device /dev/kvm` should carry a rootless-podman fallback to `-v /dev/kvm:/dev/kvm` plus a `group-add` for the `kvm` gid.

Headless GPU (verified): the SDK emulator's headless-safe configuration is `-no-window -gpu swiftshader_indirect` (SwiftShader = software GL, no host GPU dependency); `-no-accel -gpu swiftshader` is the fully-software fallback when KVM is unavailable (codersera.com/blog/android-emulator-docker-without-kvm). Host-GPU passthrough into a container is not reliable across drivers; `swiftshader_indirect` is the reproducible choice for a gate. Keep CPU acceleration (KVM) and GPU rendering (SwiftShader) as separate concerns — KVM accelerates the guest CPU, SwiftShader handles GL without the host GPU.

adb-over-TCP wiring (verified): the container publishes 5555 (adb) and (for the Google scripts) 8554/8555 (gRPC); the host connects with `adb connect localhost:5555` (github.com/google/android-emulator-container-scripts). Google's images also expose a gRPC service to talk to the running emulator inside the container.

Snapshot pitfall (matches Lava §6.I clause 6): cold-boot is mandatory for gate runs; snapshot reuse is a known false-green source. This is an operational discipline, not an image feature — Lava already encodes it constitutionally.

Concrete OSS projects (all fit Lava because they produce a real ADB-reachable Android over TCP that connectedAndroidTest and Midscore can drive):

Project	Repo	License	Recency	KVM	Notes / why it fits
android-emulator-container-scripts	https://github.com/google/android-emulator-container-scripts	Apache-2.0	368 commits; last-commit date UNVERIFIED from listing	Yes -- device / dev/kvm	Official Google; builds images from emulator+system-images ZIPs via emu-docker create; gRPC + adb Best fidelity-to-CI possible GPU HW accel documented for NDK only. (repo)
budtmo/docker-android	https://github.com/budtmo/docker-android	MIT	v3.5.2-p0 (2026-06-24), 112 releases, 15.5k stars	Yes -- device / dev/kvm	noVNC (WEB_VNC=true) API 28/29/30/32/33 adb connect from outside. Video recording + headless are more a "Pro" sponsor tier the current README (UNVERIFIED whether OSS video remains in tree). (repo)
HQarroum/docker-android	https://github.com/HQarroum/docker-android	UNVERIFIED (check LICENSE)	UNVERIFIED	Yes	Size-optimized, emulator+adb port forwarded on container NIC, headless for CI farms. (repo)
redroid (remote-android)	https://github.com/remote-android/redroid-doc	Apache-2.0 (kernel modules GPLv2)	96 commits; last-commit date UNVERIFIED	No — uses host kernel binder/ashmem, never / dev/kvm	Android 8.1–16, `redroid_gpu_mod`
thedrhax/android-avd, bazted/docker-android	https://github.com/thedrhax/dockerfiles/android-avd ,	UNVERIFIED	UNVERIFIED	Yes	Minimal headless AVD for instrumentation tests, adb 5554/5555 (repo)

Project	Repo	License	Recency	KVM	Notes / why it fits
android-avd	https://github.com/bazted/docker-android-avd				

2. Real-time full video recording that survives restarts and long (>3 min) sessions

Root cause of the 3-minute limit (verified): Android's `adb shell screenrecord` caps at 180s via a hardcoded constant `static const uint32_t kMaxTimeLimitSec = 180;` in the AOSP `screenrecord` binary (maestro.dev/fixing-androids-3-minute-screen-recording-limitation). Two production-grade escapes:

- 1. Binary rewrite (Maestro's approach, verified):** locate the little-endian `b4000000` (=180) in the compiled `screenrecord`, overwrite with a larger limit (e.g. 3600) using `Radare2` (`wx`), and push the patched binary onto the device pool. Produces one uninterrupted file, no stitch seams, works across Android versions (maestro.dev).
- 2. scrcpy (verified, recommended for Lava):** `scrcpy` has no 3-minute cap and records to `.mp4/.mkv` via `--record-format` (manpages.debian.org/testing/scrcpy/scrcpy.1.en.html). It reads the device H.264/H.265 stream over ADB and muxes host-side, so recording is decoupled from the app process and survives app restarts within one session. `scrcpy`'s own author notes it does not suffer the `screenrecord` time limit (github.com/Genymobile/scrcpy/issues/1315). Repo: <https://github.com/Genymobile/scrcpy> (Apache-2.0). `scrcpy` can record headless (no mirror window) — suitable for a container.

Chunk-and-stitch alternative (verified tradeoff): looping `screenrecord` to emit successive `mp4` files and stitching with `ffmpeg` introduces a skip at each seam (medium.com/@leland_40592/...). Prefer `scrcpy` or the binary rewrite over this for gate evidence.

Recording backends compared:

- `adb screenrecord`: on-device, 180s cap, H.264, dies when the app/session resets. Necessary-but-limited.

- `scrcpy`: host-side capture over ADB, no cap, .mp4/.mkv, headless-capable, Apache-2.0. Best default.
- `ffmpeg x11grab`: captures a host X display — only relevant if the emulator renders to a host X server (noVNC/Xvfb path in `budtmo/docker-android`); couples the recording to the display server rather than the device.
- Emulator gRPC/console recording: Google's container scripts expose a gRPC service to the running emulator (github.com/google/android-emulator-container-scripts); the emulator also supports console-driven recording. Exact long-session behavior UNVERIFIED.
- `hansalemaos/adbnativeblitz`: wraps `screenrecord` with reduced per-frame subprocess overhead for faster continuous capture, MIT-ish (verify) (github.com/hansalemaos/adbnativeblitz). Niche.

Tamper-evident chunking + hashing (verified primitives): FFmpeg's segment muxer splits a stream into sequential files (ffmpeg.org/ffmpeg-formats.html); FFmpeg's hash and framehash muxers compute cryptographic digests (MD5/SHA-256/etc.) over stream/frame data (manpages.debian.org/stretch/ffmpeg/ffmpeg-formats.1.en.html). Recommended Lava recipe (matches §11.4.128 raw-layout + curated-evidence discipline): record with `scrcpy` → segment into N-second chunks with the segment muxer → sha256sum each chunk → write a manifest where `chunk[i].prev_hash = sha256(chunk[i-1])` (a hash chain), so any single-chunk tampering breaks the chain. FFmpeg has no built-in chain-of-custody chaining; the chain is a thin shell wrapper the Containers submodule owns. UNVERIFIED: any single OSS tool that does the full record+segment+chain in one step (build it as Lava glue).

3. AI vision-analysis of test recordings to verify user-visible outcomes

Two families exist: (a) pixel/perceptual diff tools (BackstopJS, Argos, Aye Spy, Playwright visual comparisons) that catch regressions vs a golden baseline, and (b) VLM-as-oracle tools that answer natural-language assertions ("search results rendered", "download link visible") from a screenshot/frame — the latter matches Lava's §6.AB "rendering correctness not just presence" requirement and needs no golden baseline.

VLM-as-oracle (recommended for Lava's semantic assertions):

Tool	Repo	License	Recency	Why it fits Lava
Midscene.js	https://github.com/web-infra-dev/midscene	MIT	authored 2025, active	Drives Android over ADB ; screenshot-only, no selectors; <code>aiAssert("search results are visible")</code> natural-language oracle; supports self-hostable VLMs (UI-TARS, Qwen3.x, GLM-4.6V) so no cloud dependency and no \$6.H credential leak to a SaaS. Directly usable as the vision-assertion step over recorded frames or live device. (repo , midscenejs.com/introduction)
UI-TARS (models)	https://github.com/bytedance/UI-TARS	Apache-2.0	UI-TARS-1.5 / 2 in 2025, ~27–32k stars	Open-weights vision-language GUI model, sizes 2B / 7B / 72B , trained for screen understanding + action prediction. Self-host the 2B/7B on the gate host for offline frame verification. (repo)
UI-TARS-desktop / Agent-TARS	https://github.com/bytedance/	Apache-2.0	active 2025–2026	Agent stack + CLI wrapping the models; reference for the

Tool	Repo	License	Recency	Why it fits Lava
	UI-TARS-desktop			sample→assert loop. (repo)

Perceptual/diff baseline (complementary regression signal):

BackstopJS, Argos, Aye Spy are OSS (percy.io/blog/open-source-visual-regression-testing-tools, browserstack.com/guide/visual-regression-testing-open-source). They are web-oriented; for Android use them on captured screenshots. The 2026 industry trend is "Vision AI catches what pixel-diff misses" (semantic noise filtering, ignoring dynamic banners/animations) (dev.to/drizzdev/mobile-visual-regression-testing-in-2026-...).

Frame-sampling + assertion loop (how teams structure it): the standard structure is: (1) record with scrcpy; (2) sample keyframes at scene changes or fixed cadence (`ffmpeg select='gt(scene,0.3)' or fps=1`); (3) feed each sampled frame + a natural-language assertion to the VLM; (4) require a structured JSON verdict (`{pass: bool, evidence: "...", confidence}`); (5) treat the VLM verdict as a secondary CI signal that must agree with the Espresso/Compose assertion — divergence flags a bluff. This mirrors Lava's \$6.AK "gate-coverage must intersect cycle-claims": the VLM verifies the same user-visible outcome the Challenge claims. UNVERIFIED: a single turnkey OSS "video→assertion CI gate" for Android; assemble from Midscene + ffmpeg sampling.

4. Legal authorized-testing extraction of .torrent/magnet from RuTracker/RuTor/NNM-Club/Kinozal behind Cloudflare

Protection reality (verified): RuTracker.org and NNM-Club use Cloudflare ([forum.old-dos.ru viewtopic t=2024](https://forum.old-dos.ru/viewtopic?t=2024), lightrun.com/answers/...rutracker...cloudflare); DDoS-Guard is a separate Russian anti-DDoS layer also present in this ecosystem; Kinozal is proxy-sensitive and returns 403 with its DDoS protection when the egress looks wrong (github.com/Lifailon/TorAPI). RuTracker also requires an authenticated session (login cookie) for .torrent download — the challenge is Cloudflare cookie + auth cookie, not just Cloudflare.

Direct-match OSS (highest value):

- **TorAPI** — <https://github.com/Lifailon/TorAPI> — MIT, Node.js/Express, last release 0.5.2 (2024-10-09). Unofficial API + RSS for **exactly RuTracker (3 mirrors), Kinozal (3), RuTor (2), NoNameClub (1)**; returns **both .torrent files and magnet links/hashes**; search by title/ID, category filter; deploys via Docker/NPM/Vercel. Why it fits Lava: it is a ready reference implementation for the same four trackers Lava targets, usable as an oracle to cross-check Lava's own parser output. Its Cloudflare handling is UNVERIFIED from the README (no stated headless-browser/proxy mechanism). ([repo](#))
- **imDMG/qBt_SE** and **nbusseneau/qBittorrent-RuTracker-plugin** — qBittorrent search plugins for these trackers; the RuTracker plugin's open issue #41 documents the Cloudflare-block failure mode and community fixes (github.com/imDMG/qBt_SE, github.com/nbusseneau/qBittorrent-RuTracker-plugin/issues/41).

TLS-fingerprint HTTP clients (defeat JA3/JA4 blocks without a browser):

- **curl_cffi** — https://github.com/lexiforest/curl_cffi — MIT, v0.15.1b2 (2026-06), 6k+ stars, 534 commits. Impersonates Chrome/Firefox/Safari **JA3/TLS + HTTP2**, custom ja3=/akamai=/extra_fp=, **HTTP/3 + UDP socks5** since v0.15.0, asyncio + per-request proxy rotation, curl-cffi update to refresh fingerprints without upgrading. Why it fits Lava: lightweight, no browser, correct TLS fingerprint + proper browser headers is the legal-authorized-testing baseline; pair with an authenticated session cookie for RuTracker. ([repo](#))
- Underlying: **curl-impersonate** (the fork curl_cffi binds). Utlis (Go) is the equivalent for Go clients — relevant if wiring into lava-api-go (scrapfly.io/blog/posts/curl-impersonate-scrape-chrome-firefox-tls-http2-fingerprint).

Browser-based challenge solvers (for full JS/Turnstile challenges):

- **FlareSolvrr** — <https://github.com/FlareSolvrr/FlareSolvrr> — latest v3.4.6 (2025-11-29). Selenium + undetected-chromedriver proxy that solves the challenge and returns solution cookies; built-in integration in Prowlarr/Jackett (trash-guides.info/Prowlarr/prowlarr-setup-flaresolvrr). **Caveat (verified)**: upstream signals deprecation / reduced maintenance and it does not solve CAPTCHAs; it clears light JS challenges but breaks on Cloudflare Turnstile / 2026 Managed Challenges (godberrystudios.com/posts/byparr-scraping-flaresolvrr-cloudflare-bypass-2026).

- **Byparr** — <https://github.com/ThePhaseless/Byparr> — **drop-in FlareSolvrr replacement** (same API + port), v2.x uses **Camoufox** (Firefox-based anti-detect, C++ fingerprint patches), v2.1.0 (2026-02) added Python 3.14 + per-request X-Proxy-* overrides, weekly-ish release cadence. License UNVERIFIED (check repo). Why it fits Lava: swap one base URL vs FlareSolvrr; better against modern Cloudflare. ([repo](#))
- **Byparr-nodriver** (<https://github.com/hatemosphere/Byparr-nodriver>, seleniumbase/nodriver + FastAPI), **trawl** (<https://github.com/germondai/trawl>, self-hosted, targets Turnstile/reCAPTCHA/hCaptcha/GeeTest), **Solvearr** (<https://github.com/nabilak/Solvearr>, lightweight no-browser, *arr-compatible) are alternatives ([search results](#)).

Egress (legal note): residential/VPN egress reduces DDoS-Guard/Kinozal 403s (Kinozal is proxy-sensitive per TorAPI docs). For Lava this is authorized testing of the operator's own accounts; keep credentials in .env per §6.H and never in the scraper source per §6.R. Tradeoff table:

Layer	Tool	Defeats	Cost	Fidelity
TLS/JA3 only	curl_cffi / curl-impersonate / utls	JA3/JA4/HTTP2 fingerprint blocks	low (no browser)	high for header/TLS, none for JS challenges
Full JS challenge	Byparr (Camoufox) / FlareSolvrr	Cloudflare IUAM/JS, returns cookies	medium (headless browser RAM)	high, Turnstile UNVERIFIED
Ready API	TorAPI	all-of-the-above for these 4 trackers, if it works	medium	direct-match oracle for Lava

5. Game-changing / novel approaches worth flagging

- **Google Cuttlefish (cvd)** — **self-hostable "AWS Device Farm equivalent" (verified)**. Configurable virtual Android device, runs locally (x86 + ARM64) or in cloud; official on-prem path is the

Cuttlefish Docker image + Cloud Orchestrator web service; couples with Kubernetes for elastic fleets (source.android.com/docs/devices/cuttlefish/on-premises, realz.medium.com/running-android-on-kubernetes). Repos: <https://github.com/google/android-cuttlefish> (Apache-2.0), community <https://github.com/thatoddm mailbox/cuttlefish-docker>. Needs KVM + vhost_vsock/vhost_net. This is the highest-fidelity self-hosted device-farm primitive and a strong candidate to back Lava's \$6.AE matrix at scale (multiple API levels as k8s pods). It is still KVM-in-a-container (not fully isolated) (source.android.com/docs/devices/cuttlefish/on-premises).

- **redroid microVM-free path (verified).** Real Android userspace on the host kernel (binder/ashmem), no QEMU, no /dev/kvm — the fastest headless option on runners lacking nested virt (github.com/remote-android/redroid-doc). Novel for cost, but device-parity with the SDK AVD is UNVERIFIED for gate fidelity.
- **DeviceFarmer / STF (verified, caveated).** OSS continuation of OpenSTF: browser control of real device fleets, REST API for CI, booking/partitioning; v3.7.7 (2025-07-23); **volunteer-maintained, slow progress** (github.com/DeviceFarmer/stf, medium.com/@jerimiahham/building-device-farms-with-stf). Fits Lava if physical devices are added later; heavier than needed for emulator-only gates.
- **Genymotion Device Image / PaaS (verified, non-OSS).** Android AMIs on AWS/GCP/Alibaba/Azure, per-minute-per-device pricing, plus on-prem; ARM64 images up to Android 15 (genymotion.com/product-device-image, aws.amazon.com/marketplace ... [Genymotion PaaS Android 15](https://genymotion.com/paas-android-15)). Not OSS and cloud-cost-bearing — conflicts with Lava's Local-Only CI/CD and \$6.W posture; flag as a fallback only.
- **Maestro for Compose (verified, Apache-2.0).** Black-box UI automation that reads Compose semantics automatically and prioritizes text/semantics over testTag; testTagsAsResourceId = true exposes tags; maestro hierarchy dumps targets (docs.maestro.dev/platform-support/android-jetpack-compose). Novel vs Espresso: no instrumentation coupling, YAML flows, resilient to refactors — a lighter complement (not replacement) to Lava's Compose Challenge Tests, and the source of the screenrecord binary-rewrite technique in §2.
- **Midscene + self-hosted VLM as an appium alternative (verified).** Natural-language, selector-free Android automation over ADB with local UI-TARS/Qwen — no Appium server, no brittle locators (github.com/web-infra-dev/midscene). This is the "vision agent drives the app AND verifies it" convergence of §3 and §5.

- **UNVERIFIED to flag, not adopt:** gVisor does not run Android guests (it is a syscall sandbox for Linux containers), so it is not an emulator path; QEMU-full-system Android microVMs beyond Cuttlefish/AVF have no mature OSS project found in this pass — mark as UNKNOWN.
-

Top 5 to trial next (prioritized, with Lava-stack integration notes)

1. **Google android-emulator-container-scripts under rootless podman + KVM (Containers submodule).** Highest-fidelity, matches Lava's SDK-AVD gate today. Integration: build images with `emu-docker create` for API 28/30/34/latest; in `pkg/emulator/containerized.go` add the rootless-podman fallback `-v /dev/kvm:/dev/kvm + --group-add keep-groups` (per [podman#16701](#)); launch with `-no-window -gpu swiftshader_indirect`; adb connect on published 5555; run `connectedDebugAndroidTest`. Closes the §6.X/§6.AH "emulator inside a container" requirement on the Linux x86_64 gate host.
2. **scrcpy as the evidence recorder + ffmpeg segment/hash chunking (Containers submodule glue).** Replaces adb screenrecord to remove the 180s cap and survive app restarts. Integration: run `scrcpy --no-window --record-format=mkv` against the container's adb; pipe/segment into N-second chunks with `ffmpeg segment muxer`; sha256sum each chunk into a hash-chained manifest under the §11.4.128 raw layout `<root>/YYYY-MM-DD/<state-hash>/<DEVICE>_<SERIAL>/recording_NNN/`. Satisfies §11.4.128 always-on recording + tamper-evident evidence.
3. **Midscene.js + self-hosted UI-TARS-7B as the vision oracle (new : api-app / CI step).** Feeds sampled frames from #2 to a local VLM and asserts user-visible outcomes ("search results rendered", "download link visible") as a secondary CI signal that must agree with the Compose Challenge — directly enforcing §6.AK coverage-intersection and §6.AB rendering-correctness. Integration: run UI-TARS-7B on the gate host (Apache-2.0, offline, no §6.H leak); Midscene drives via the same adb-over-TCP as #1; emit `{pass,evidence,confidence}` JSON into the evidence dir.
4. **TorAPI + curl_cffi as the tracker-extraction oracle and hardened client (lava-api-go / test bench).** TorAPI (MIT, same four trackers) becomes a cross-check oracle for Lava's own `.torrent/` magnet parsers; `curl_cffi` (or Go `utls` for `lava-api-go`) supplies correct JA3/JA4/HTTP2 + browser headers for authorized-account

testing. Integration: stand up TorAPI in podman on the LAN; add a parity test comparing Lava's extracted magnet/hash for a known topic against TorAPI's; put credentials in .env per §6.H. Add Byparr (Camoufox) behind it only when a full JS challenge blocks the TLS-only path.

5. **Cuttlefish on-prem (Docker image + Cloud Orchestrator) for the scaled §6.AE matrix.** When the single-AVD path is proven, move to Cuttlefish for elastic multi-API/multi-form-factor fleets as k8s/podman pods, still KVM-accelerated and fully self-hosted (no cloud spend, satisfies Local-Only CI/CD). Integration: pin google/android-cuttlefish images, drive via the Cloud Orchestrator REST API from Lava-side glue, keep cold-boot-only per §6.I clause 6.

Sources (primary): [google/android-emulator-container-scripts](#) · [budtmo/docker-android](#) · [remote-android/redroid-doc](#) · [podman#16701](#) · [Red Hat rootless KVM/QEMU](#) · [emulator-without-kvm/swiftshader](#) · [Maestro screenrecord fix](#) · [Genymobile/scrcpy](#) · [scrcpy manpage](#) · [ffmpeg-formats hash/segment](#) · [web-infra-dev/midscene](#) · [bytedance/UI-TARS](#) · [percy open-source VRT](#) · [Lifailon/TorAPI](#) · [lexiforest/curl cffi](#) · [FlareSolvrr](#) · [ThePhaseless/Byparr](#) · [Byparr vs FlareSolvrr 2026](#) · [TRaSH FlareSolvrr+Prowlarr](#) · [Cuttlefish get-started](#) · [Cuttlefish on-premises](#) · [google/android-cuttlefish](#) · [DeviceFarmer/stf](#) · [Genymotion Device Image](#) · [Maestro Compose docs](#)